



Project Report:

Real-Time American Sign Language (ASL Detection APP)

Group Members:

Syed M. Kashaaf Haider	BAI241056
Syed M. Hasnain Sherazi	BAI241068
Syed M. Razi	BAI241037
Afnan Ijaz	BAI241064

Submitted To:

Dr. Saifullah Saifi

Program:

BS(Artificial Intelligence)

Subject:

Artificial Intelligence

Date:

26th June, 2025

Real-Time American Sign Language (ASL) Detection APP

1. Project Title and Overview

Title

Real-Time American Sign Language Detection Using Machine Learning and Computer Vision

Overview

This project aims to develop a real-time American Sign Language (ASL) detection system that translates hand signs into corresponding English letters. Users can sign letters (A–Z) into a webcam, and the system processes the hand landmarks and predicts the corresponding letter. It utilizes MediaPipe for real-time hand landmark detection, TensorFlow/Keras for gesture classification, and OpenCV for live video streaming. The application is designed for low-latency inference, providing an accessible communication interface for ASL users.

2. Problem Statement and Project Purpose

Problem Statement

American Sign Language is an essential communication tool for Deaf and Hard-of-Hearing individuals. However, most computer systems and people do not understand ASL, creating communication gaps and reducing access to information and services.

Purpose of the Project

This project addresses this issue by creating a system that instantly translates ASL hand signs into text. The goal is to bridge communication barriers and make it easier for Deaf and Hard-of-Hearing individuals to communicate with people who do not understand sign language.

3. Dataset Details and Training Challenges

Dataset Description

The training dataset contains over 90,000 labeled images of American Sign Language alphabet signs. Images feature diverse lighting conditions, backgrounds, and hand positions, allowing the model to generalize well across real-world variations.

Training Challenges

Training on the entire dataset at once was highly time-consuming, exceeding two days per session. This required stable internet connectivity and a continuously powered-on system, which was impractical for long-duration training. Memory constraints and risk of crashes further complicated this process. Overfitting also occurred when training on the entire dataset in one go.

Solution

To address these constraints:

- The dataset was divided into six subsets of 500 images per class.
- Training was conducted incrementally across these subsets.
- This approach reduced resource requirements and enabled stable training.
- Incremental fine-tuning helped prevent overfitting and enhanced accuracy.



4. Model Architecture and Training Process

Model Type

A Multi-Layer Perceptron (MLP) was selected due to its efficiency for numeric data such as hand landmarks.

Input Features

Each input vector contains 63 numeric features corresponding to the x, y, z coordinates of 21 MediaPipe landmarks.

Model Architecture

- Input Layer: 63 neurons
- Hidden Layer 1: Dense(128), ReLU
- Hidden Layer 2: Dense(64), ReLU
- Output Layer: Dense(26), Softmax

Training Details

- Loss function: Categorical cross-entropy
- Optimizer: Adam

Project Report

- Epochs per subset: 10–30
- Batch size: 32 or 64
- Saved model file: asl_mlp_final_trained.keras

Training Platforms

Training was conducted on both Kaggle and Google Colab platforms to leverage their cloud GPU/TPU resources. This ensured faster training, better performance tuning, and avoided the need for long-duration training on a local machine.

Output

The trained model outputs a probability vector for all 26 classes. The class with the highest probability is selected as the predicted letter.

Model Training Code:

```
# 🟢 Final Training on Dataset6 with Improvements (MLP + Hand Landmarks)
import os, cv2, json
import numpy as np
import pandas as pd
import mediapipe as mp
import matplotlib.pyplot as plt
from tqdm import tqdm
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import load_model, Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
from tensorflow.keras.utils import to_categorical

# 📁 Paths
DATASET_DIR = "/content/extracted_data/project_AI_dataset6/dataset6"
PREVIOUS_MODEL_PATH = "asl_mlp_dataset5.keras"
CLASS_INDEX_PATH = "class_indices_dataset1.json"
MODEL_SAVE_PATH = "asl_mlp_final_trained.keras"

# 📄 Load model and label mapping
model = load_model(PREVIOUS_MODEL_PATH)
with open(CLASS_INDEX_PATH, 'r') as f:
    label_map = json.load(f)
    class_names = [label_map[str(i)] for i in range(len(label_map))]

# 🖐️ Extract Hand Landmarks from Dataset6
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(static_image_mode=True, max_num_hands=1)
X, y = [], []
for label in class_names:
    folder = os.path.join(DATASET_DIR, label)
    if not os.path.exists(folder): continue
    for img_name in tqdm(os.listdir(folder), desc=f"Extracting {label}"):
        img_path = os.path.join(folder, img_name)
        img = cv2.imread(img_path)
        if img is None: continue
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        results = hands.process(img_rgb)
        if results.multi_hand_landmarks:
            lm = results.multi_hand_landmarks[0]
            features = [coord for point in lm.landmark for coord in (point.x, point.y, point.z)]
            X.append(features)
            y.append(label)
hands.close()

# 🎨 Encode & Normalize
label_index = {v: int(k) for k, v in label_map.items()}
y_encoded = [label_index[lbl] for lbl in y]
y_cat = to_categorical(y_encoded, num_classes=len(label_map))
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 📊 Split
X_train, X_val, y_train, y_val = train_test_split(X_scaled, y_cat, test_size=0.1, random_state=42, stratify=y_encoded)

# 🔄 Continue training with improvements
callbacks = [
    ModelCheckpoint(MODEL_SAVE_PATH, save_best_only=True, monitor='val_loss', mode='min'),
    EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
]
history = model.fit(X_train, y_train, validation_data=(X_val, y_val),
                    epochs=20, batch_size=64, callbacks=callbacks)

# 📄 Save model
print(f"🟢 Final training complete. Model saved to: {MODEL_SAVE_PATH}")

# 📊 Plot Results
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title("Final Accuracy")
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title("Final Loss")
plt.legend()
plt.tight_layout()
plt.show()
```

Training Processing and Output:

```

Extracting A: 100%|██████████| 500/500 [00:16<00:00, 29.65it/s]
Extracting B: 100%|██████████| 500/500 [00:15<00:00, 31.28it/s]
Extracting C: 100%|██████████| 500/500 [00:14<00:00, 35.03it/s]
Extracting D: 100%|██████████| 500/500 [00:16<00:00, 30.96it/s]
Extracting E: 100%|██████████| 500/500 [00:16<00:00, 31.01it/s]
Extracting F: 100%|██████████| 500/500 [00:22<00:00, 21.85it/s]
Extracting G: 100%|██████████| 500/500 [00:18<00:00, 27.61it/s]
Extracting H: 100%|██████████| 500/500 [00:15<00:00, 33.28it/s]
Extracting I: 100%|██████████| 500/500 [00:15<00:00, 32.64it/s]
Extracting J: 100%|██████████| 500/500 [00:17<00:00, 28.69it/s]
Extracting K: 100%|██████████| 500/500 [00:20<00:00, 23.82it/s]
Extracting L: 100%|██████████| 500/500 [00:17<00:00, 29.35it/s]
Extracting M: 100%|██████████| 500/500 [00:15<00:00, 32.85it/s]
Extracting N: 100%|██████████| 500/500 [00:14<00:00, 34.17it/s]
Extracting O: 100%|██████████| 500/500 [00:15<00:00, 32.17it/s]
Extracting P: 100%|██████████| 500/500 [00:13<00:00, 36.96it/s]
Extracting Q: 100%|██████████| 500/500 [00:14<00:00, 34.04it/s]
Extracting R: 100%|██████████| 500/500 [00:20<00:00, 24.47it/s]
Extracting S: 100%|██████████| 500/500 [00:18<00:00, 26.36it/s]
Extracting T: 100%|██████████| 500/500 [00:21<00:00, 22.89it/s]
Extracting U: 100%|██████████| 500/500 [00:17<00:00, 28.65it/s]
Extracting V: 100%|██████████| 500/500 [00:18<00:00, 27.50it/s]
Extracting W: 100%|██████████| 500/500 [00:20<00:00, 24.46it/s]
Extracting X: 100%|██████████| 500/500 [00:15<00:00, 33.06it/s]
Extracting Y: 100%|██████████| 500/500 [00:15<00:00, 31.31it/s]
Extracting Z: 100%|██████████| 500/500 [00:16<00:00, 30.38it/s]
Extracting del: 100%|██████████| 500/500 [00:14<00:00, 33.51it/s]
Extracting space: 100%|██████████| 500/500 [00:14<00:00, 33.92it/s]
Epoch 1/20

```

```

Epoch 1/20
63/63 ————— 2s 7ms/step - accuracy: 0.8587 - loss: 0.5713 - val_accuracy: 0.9459 - val_loss: 0.2244
Epoch 2/20
63/63 ————— 0s 7ms/step - accuracy: 0.9064 - loss: 0.3372 - val_accuracy: 0.9527 - val_loss: 0.2022
Epoch 3/20
63/63 ————— 0s 7ms/step - accuracy: 0.9131 - loss: 0.2531 - val_accuracy: 0.9685 - val_loss: 0.1823
Epoch 4/20
63/63 ————— 1s 7ms/step - accuracy: 0.9273 - loss: 0.2538 - val_accuracy: 0.9662 - val_loss: 0.1710
Epoch 5/20
63/63 ————— 1s 7ms/step - accuracy: 0.9343 - loss: 0.2074 - val_accuracy: 0.9730 - val_loss: 0.1641
Epoch 6/20
63/63 ————— 1s 7ms/step - accuracy: 0.9350 - loss: 0.1944 - val_accuracy: 0.9707 - val_loss: 0.1562
Epoch 7/20
63/63 ————— 1s 6ms/step - accuracy: 0.9365 - loss: 0.1816 - val_accuracy: 0.9752 - val_loss: 0.1574
Epoch 8/20
63/63 ————— 0s 4ms/step - accuracy: 0.9406 - loss: 0.1779 - val_accuracy: 0.9752 - val_loss: 0.1383
Epoch 9/20
63/63 ————— 0s 4ms/step - accuracy: 0.9454 - loss: 0.1659 - val_accuracy: 0.9730 - val_loss: 0.1464
Epoch 10/20
63/63 ————— 0s 4ms/step - accuracy: 0.9477 - loss: 0.1776 - val_accuracy: 0.9820 - val_loss: 0.1421
Epoch 11/20
63/63 ————— 0s 4ms/step - accuracy: 0.9497 - loss: 0.1714 - val_accuracy: 0.9842 - val_loss: 0.1530
Epoch 12/20
63/63 ————— 0s 4ms/step - accuracy: 0.9490 - loss: 0.1466 - val_accuracy: 0.9865 - val_loss: 0.1497
Epoch 13/20
63/63 ————— 0s 4ms/step - accuracy: 0.9566 - loss: 0.1561 - val_accuracy: 0.9640 - val_loss: 0.1670

✅ Final training complete. Model saved to: asl_mlp_final_trained.keras

```

5. Technologies and Libraries used in Model Training:

Library/Tool	Purpose	Explanation
OpenCV (cv2)	Capture live video and process image frames.	Enables stable and efficient real-time webcam input and image manipulation.
MediaPipe	Real-time hand landmark detection.	Provides accurate, lightweight hand-tracking and landmark extraction without custom training.
TensorFlow/Keras	Model training and inference.	Offers intuitive APIs for building and training deep learning models and seamlessly deploying them.
NumPy	Numeric computations.	Efficient handling of numeric arrays for input to the model.
JSON	Load label mappings.	Enables easy mapping between class indices and ASL letters.
Time	Measure performance.	Tracks processing times per frame to optimize the real-time experience.

6. Training Data Preprocessing and Feature Extraction

Purpose

Preprocess raw images into normalized numeric feature vectors that match the model input format.

Preprocessing Steps

1. Read label mappings (class_indices_dataset1.json).
2. Use MediaPipe to extract x, y, z landmarks from each image.
3. Store the flattened 63-length vectors in a list.
4. Normalize features with StandardScaler() to improve training convergence.
5. Save the scaler as scaler_dataset6.pkl to ensure consistent normalization at inference.

Error Handling

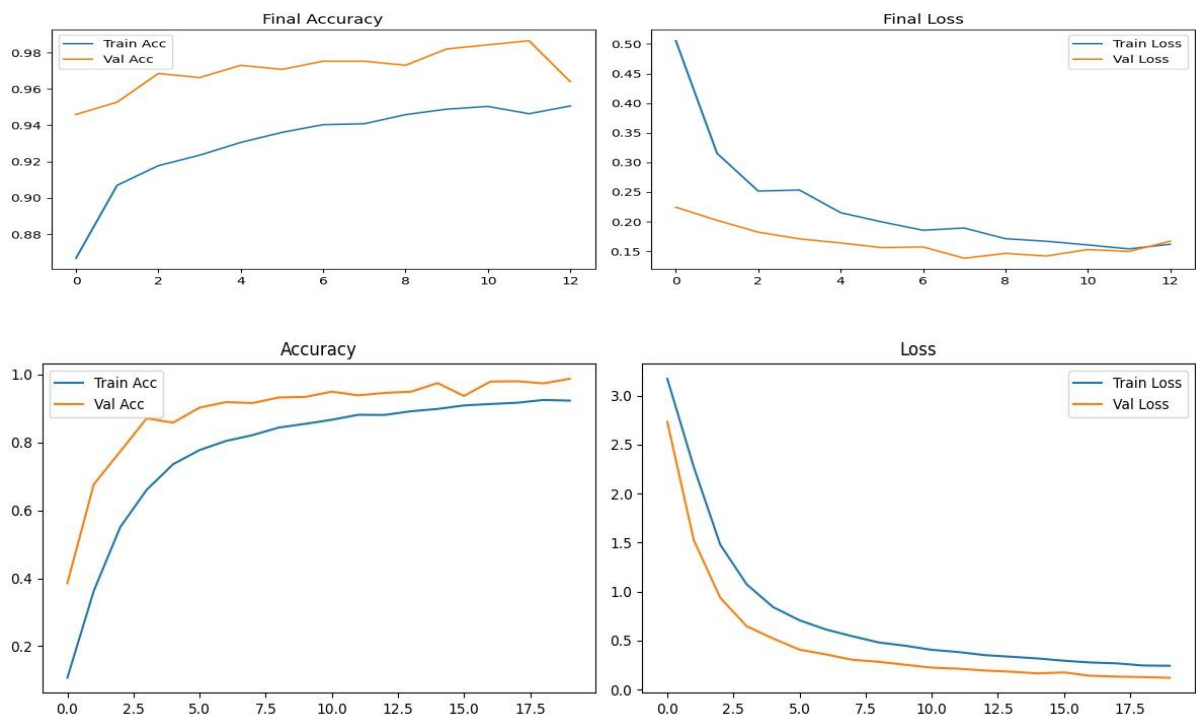
Checks ensure all files exist and landmarks are correctly detected. Any corrupted or unreadable images are logged.

Outcome

Every image is successfully processed into a normalized feature vector that can be fed into the model.

Significance

Consistent normalization improves training stability and predictive accuracy. This same scaler is reused during live inference to maintain the training-inference feature distribution.



7. Live Sign Language Translator GUI Application

Title

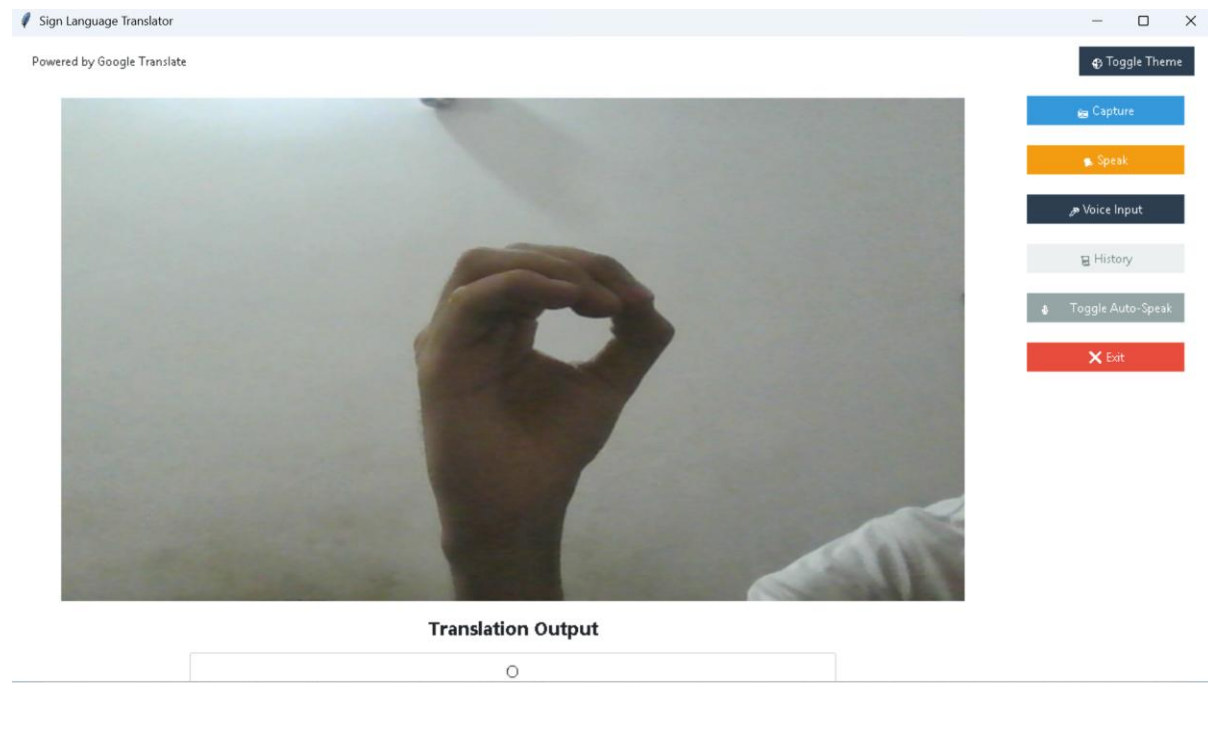
Live Sign Language Translator Application with Auto-Speak and Multi-Language Support

Introduction

This GUI-driven application uses a trained deep learning model to recognize letters in real time. It also provides text output, speech output, language translation, and a history log for a fully interactive experience.

Application Flow

- **Start screen** – User agrees to webcam access.
- **Main interface** –
 - Left panel: Live video preview and detected letter.
 - Right panel: Buttons for Capture, Speak, Voice Input, History, Auto-Speak Toggle, Theme Toggle.
- **Processing**
 - Capture frames continuously.
 - Extract landmarks.
 - Predict the corresponding letter.
 - Translate into the selected language.
 - Speak output if enabled.



8. GUI Application Technologies and Libraries

Library/Tool	Purpose	Explanation
Tkinter tkbootstrap	& Build and style the graphical interface.	Provides accessible widgets and themes for a user-friendly desktop application.
MediaPipe	Detect hand landmarks.	Real-time landmark detection powers the recognition process.
TensorFlow/Keras	Run the trained model for inference.	Enables accurate, fast letter predictions.
Googletrans	Translate detected letters into different languages.	Supports multilingual communication for diverse users.
pyttsx3	Convert detected output to speech.	Offers offline, real-time text-to-speech functionality.
speech_recognition	Capture voice input for hands-free control.	Enables speech-driven commands and hands-free app interactions.
PIL (Pillow)	Handle and display image frames inside the GUI.	Converts frames into a format compatible with the UI.
OpenCV (cv2)	Capture and process webcam frames.	Streams the live video feed and handles color and image manipulation.
NumPy	Efficient numeric operations.	Transforms landmarks into model input arrays.
JSON	Load label mappings and configurations.	Retrieves class mappings to decode model predictions into letters.
datetime, os	Save image files and manage file paths.	Organizes the log of detected letters with timestamps for history tracking.

GUI Code:

```

1  import cv2
2  import tkinter as tk
3  from ttkbootstrap import style, ttk
4  from tkinter import messagebox, filedialog
5  from PIL import Image, ImageTk
6  import pyttsx3
7  import datetime
8  import speech_recognition as sr
9  from googletrans import Translator
10 from tensorflow.keras.models import load_model
11 import numpy as np
12 import json
13 import os
14 import mediapipe as mp
15
16 class SignLanguageApp:
17     def __init__(self, root):
18         self.root = root
19         self.root.title("Sign Language Translator")
20         self.root.geometry("1200x800")
21         self.theme = "Flatly"
22         self.style = style(self.theme)
23
24         self.engine = pyttsx3.init()
25         self.cap = None
26         self.last_frame = None
27         self.translation_text = tk.StringVar(value="")
28         self.translation_history = []
29         self.selected_lang = tk.StringVar(value='en')
30         self.translator = Translator()
31
32         self.auto_speak_enabled = False # 🗣️ NEH: Flag for auto-speak mode
33
34         if not os.path.exists("asl_mlp_final_trained.keras"):
35             messagebox.showerror("Error", "Model file 'asl_mlp_final_trained.keras' not found!")
36             self.root.quit()
37             return
38         self.model = load_model("asl_mlp_final_trained.keras")
39
40         with open("class_indices_dataset1.json", "r") as f:
41             class_indices = json.load(f)
42         self.labels = [label for index, label in sorted(class_indices.items(), key=lambda x: int(x[0]))]
43
44         self.mp_hands = mp.solutions.hands
45         self.hands = self.mp_hands.Hands(static_image_mode=False, max_num_hands=1, min_detection_confidence=0.5)
46
47         self.start_screen()
48
49     def start_screen(self):
50         self.clean_screen()
51         self.root.configure(bg="#333333")
52         frame = ttk.Frame(self.root, padding=50)
53         frame.place(relx=0.5, rely=0.5, anchor="center")
54
55         ttk.Label(frame, text="👋 Sign Language Translator",
56                  font=("Segoe UI", 26, "bold")).pack(pady=30)
57         ttk.Label(frame, text="Click below to begin",
58                  font=("Segoe UI", 14)).pack(pady=5)
59         ttk.Button(frame, text="🚀 Start App",
60                   command=self.ask_permission,
61                   bootstyle="purple").pack(pady=20)
62
63     def ask_permission(self):
64         if messagebox.askyesno("Camera Access", "This app requires webcam. Allow?"):
65             self.setup_ui()
66
67
68     def setup_ui(self):
69         self.cap = cv2.VideoCapture(0)
70         self.show_frame()
71         self.root.quit()
72
73     def setup_ui(self):
74         self.clean_screen()
75         self.root.configure(bg="SystemButtonFace")
76
77         main_frame = ttk.Frame(self.root, padding=10)
78         main_frame.pack(fill="both", expand=True)
79
80         top_bar = ttk.Frame(main_frame)
81         top_bar.pack(fill="x", pady=(0, 10))
82         ttk.Label(top_bar, text="Powered by Google Translate").pack(side="left", padx=10)
83         ttk.Button(top_bar, text="🔄 Toggle Theme",
84                   command=self.toggle_theme).pack(side="right", padx=10)
85
86         body_frame = ttk.Frame(main_frame)
87         body_frame.pack(fill="both", expand=True)
88
89         left_frame = ttk.Frame(body_frame)
90         left_frame.pack(side="left", fill="both", expand=True, padx=10)
91
92         self.video_label = ttk.Label(left_frame)
93         self.video_label.pack(pady=10)
94
95         ttk.Label(left_frame, text="Translation Output",
96                  font=("Segoe UI", 24, "bold")).pack()
97         self.translation_display = ttk.Entry(
98             left_frame, textvariable=self.translation_text,
99             font=("Segoe UI", 13), width=70, justify="center")
100         self.translation_display.pack(pady=10)
101
102         ttk.Label(left_frame, text="Select Language:",
103                  font=("Segoe UI", 12)).pack()
104         ttk.Combobox(
105             left_frame, textvariable=self.selected_lang,
106             values=["en", "ur", "fr", "es", "zh-cn", "ar"],
107             width=20).pack(pady=5)
108
109         right_frame = ttk.Frame(body_frame)
110         right_frame.pack(side="right", fill="y", padx=20)
111
112         ttk.Button(right_frame, text="📷 Capture",
113                   command=self.capture_frame, bootstyle="info").pack(pady=10, fill="x")
114         ttk.Button(right_frame, text="🗣️ Speak",
115                   command=self.speak_translation, bootstyle="warning").pack(pady=10, fill="x")
116         ttk.Button(right_frame, text="👂 Voice Input",
117                   command=self.voice_input, bootstyle="primary").pack(pady=10, fill="x")
118         ttk.Button(right_frame, text="📜 History",
119                   command=self.show_history, bootstyle="light").pack(pady=10, fill="x")
120
121         # 🗣️ NEH Toggle Speak mode
122         ttk.Button(right_frame, text="🔊 Toggle Auto-Speak",
123                   command=self.toggle_auto_speak, bootstyle="secondary").pack(pady=10, fill="x")
124
125         ttk.Button(right_frame, text="❌ Exit",
126                   command=self.exit_app, bootstyle="danger").pack(pady=10, fill="x")
127
128     def show_frame(self):
129         if self.cap and self.cap.isOpened():
130             ret, frame = self.cap.read()

```

```

129         if ret:
130             self.last_frame = frame
131             frame_flipped = cv2.flip(frame, 1)
132             frame_flipped = cv2.resize(frame_flipped, (500, 500))
133
134             img_rgb = cv2.cvtColor(frame_flipped, cv2.COLOR_BGR2RGB)
135             results = self.hands.process(img_rgb)
136
137             predicted_letter = ""
138             if results.multi_hand_landmarks:
139                 landmarks = results.multi_hand_landmarks[0].landmark
140                 features = [coord for lm in landmarks for coord in (lm.x, lm.y, lm.z)]
141                 if len(features) == 63:
142                     features = np.array(features).reshape(1, -1)
143                     preds = self.model.predict(features)
144                     class_idx = preds.argmax(axis=1)[0]
145                     predicted_letter = self.labels[class_idx]
146
147             translated = self.translate_text(predicted_letter, self.selected_lang.get()) if predicted_letter else ""
148             self.translation_text.set(translated)
149
150             # 🗨️ Update: Append every translation to history
151             if translated:
152                 self.translation_history.append(translated)
153
154             # 🗣️ Update: Auto-speak if enabled
155             if self.auto_speak_enabled and translated:
156                 self.engine.say(translated)
157                 self.engine.runandwait()
158
159             img_pil = Image.fromarray(cv2.cvtColor(frame_flipped, cv2.COLOR_BGR2RGB))
160             imgtk = ImageTk.PhotoImage(image=img_pil)
161             self.video_label.imgtk = imgtk
162             self.video_label.configure(image=imgtk)
163
164             self.root.after(20, self.show_frame)
165
166     def translate_text(self, text, lang):
167         try:
168             return self.translator.translate(text, dest=lang).text
169         except:
170             return text
171
172     def capture_frame(self):
173         if self.last_frame is not None:
174             name = f"capture_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.png"
175             cv2.imwrite(name, self.last_frame)
176             messagebox.showinfo("Image Saved", f"Saved as {name}")
177         else:
178             messagebox.showwarning("No Frame", "Nothing to save")
179
180     def speak_translation(self):
181         text = self.translation_text.get()
182         if text:
183             self.engine.say(text)
184             self.engine.runandwait()
185         else:
186             messagebox.showinfo("No Text", "Nothing to speak")
187
188     def voice_input(self):
189         recognizer = sr.Recognizer()
190         with sr.Microphone() as source:
191             messagebox.showinfo("Voice Input", "Start speaking...")

```

```

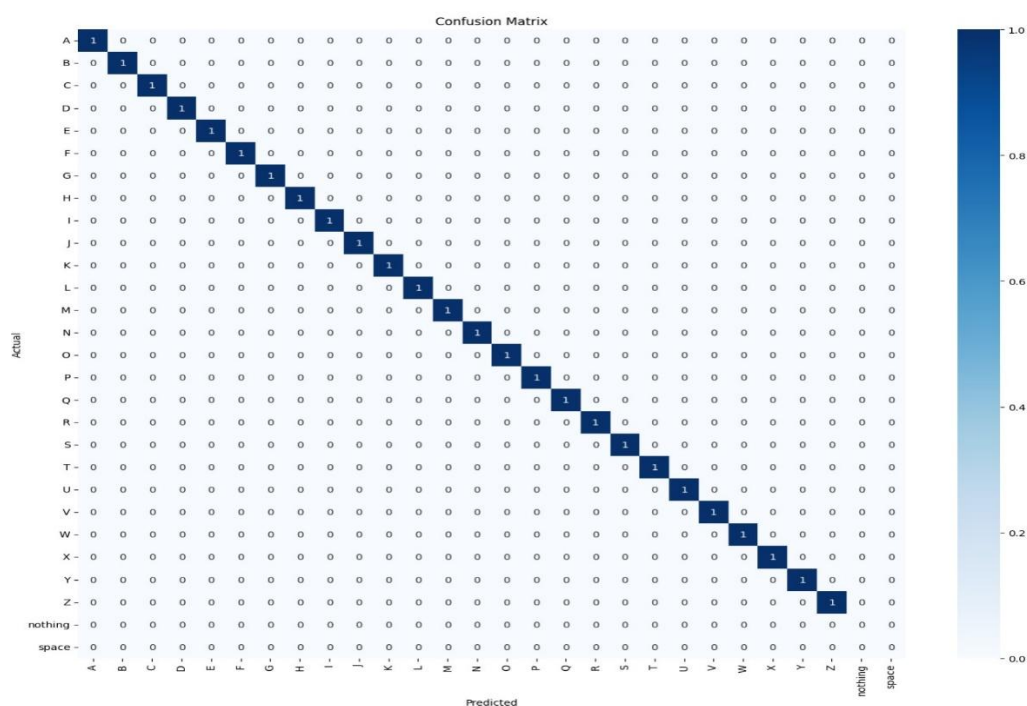
191             messagebox.showinfo("Voice Input", "Start speaking...")
192             audio = recognizer.listen(source)
193             try:
194                 text = recognizer.recognize_google(audio)
195                 self.translation_text.set(text)
196                 self.translation_history.append(text)
197             except sr.UnknownValueError:
198                 messagebox.showwarning("Voice Error", "Sorry, could not understand")
199             except sr.RequestError:
200                 messagebox.showerror("API Error", "Could not reach speech API")
201
202     def show_history(self):
203         win = tk.Toplevel(self.root)
204         win.title("Translation History")
205         win.geometry("400x300")
206         ttk.Label(win, text="Your Translations",
207                  font=("Segoe UI", 13, "bold")).pack(pady=10)
208         text_area = tk.Text(win, font=("Segoe UI", 11))
209         text_area.pack(expand=True, fill="both", padx=10, pady=10)
210         for entry in self.translation_history:
211             text_area.insert(tk.END, entry + "\n")
212         ttk.Button(win, text="Close",
213                  command=win.destroy).pack(pady=10)
214
215     def toggle_theme(self):
216         self.theme = "darkly" if self.theme == "flatly" else "flatly"
217         self.style = Style(self.theme)
218         self.setup_ui()
219
220     def toggle_auto_speak(self): # 🔄 NEW
221         self.auto_speak_enabled = not self.auto_speak_enabled
222         state = "enabled" if self.auto_speak_enabled else "disabled"
223         messagebox.showinfo("Auto-Speak Mode", f"Auto-speak {state}")
224
225     def exit_app(self):
226         if self.cap:
227             self.cap.release()
228         self.root.quit()
229
230     def clear_screen(self):
231         for widget in self.root.winfo_children():
232             widget.destroy()
233
234 if __name__ == "__main__":
235     root = tk.Tk()
236     app = SignLanguageApp(root)
237     root.mainloop()
238

```

9. Results and Achievements

The project successfully meets its goals:

- Provides real-time recognition of ASL alphabet signs.
- Translates detected signs into any supported language.
- Offers speech output and a user-friendly interface.
- Achieves stable training and high accuracy through incremental training and feature normalization.



10. Limitations and Future Work

Current Limitations

- Detects only single letters, not full words or sentences.
- Requires stable lighting and one visible hand.
- Designed for desktop usage.

Future Directions

- Expand recognition to dynamic signs and full-sentence translation.
- Develop a mobile or web version for greater accessibility.

- Explore more advanced model architectures (e.g. CNNs, LSTMs) for dynamic gestures.
 - Introduce sentence-level recognition and smoother speech output.
-

11. Conclusion

This project successfully integrates computer vision, deep learning, language translation, and speech output into a unified real-time American Sign Language translator. By leveraging robust technologies like MediaPipe, TensorFlow/Keras, Googletrans, and pyttsx3 — and training the model on distributed datasets using Kaggle and Google Colab — we created an efficient and user-friendly interface that can support inclusive communication. The architecture and features offer a strong foundation for future work toward creating an even more advanced and accessible sign language translator.